

GLR formulas as implemented (replicated backend)

The step-by-step apply/solve recipe of `lgpsf-hessian`'s `LGH_GLR_REPLICATED` backend. The ScaLAPACK backend generalizes the layouts (see `glr-distributed-design.pdf`); the mathematics is identical.

Definitions: M lumped mass (diagonal); Z the prior's (α -scaled) square-root factor (e.g. a shifted Laplacian); prior precision $R = ZM^{-1}Z$; $\tilde{H}_d$ assembled lgpsf misfit-Hessian approximation (sparse, weighted-symmetrized); c regularization/LM shift; ℓ sketch size; r kept rank; P ranks, rank p owns N_p mesh rows.

1. Low-rank eigendecomposition of the whitened misfit Hessian

$$F := M^{1/2}Z^{-1}\tilde{H}_dZ^{-1}M^{1/2} \quad (F = F^T \in \mathbb{R}^{N \times N}, \text{ never formed}) \quad (1)$$

$$\Omega_{ij} = \text{hashnormal}(\text{seed}, i, j) \quad i \in \{1, \dots, N\} \text{ global}, j \in \{1, \dots, \ell\} \quad (2)$$

$$Y \leftarrow F \Omega \quad (3)$$

$$\text{for } p = 1, \dots, q: \quad Q \leftarrow \text{cholqr2}(Y), \quad Y \leftarrow F Q \quad (4)$$

$$Q \leftarrow \text{cholqr2}(Y), \quad Y \leftarrow F Q \quad (5)$$

$$T \leftarrow \frac{1}{2} (Q^T Y + (Q^T Y)^T) \quad (\text{local GEMM} + \text{one allreduce}) \quad (6)$$

$$T = V \Lambda_{\text{raw}} V^T \quad (\text{dsyevd}, \text{ redundant on every rank}) \quad (7)$$

$$\mathcal{K} = \{i : |\lambda_{\text{raw},i}| > \tau\}, \quad r = |\mathcal{K}| \quad (\text{sorted by } |\lambda| \text{ descending}) \quad (8)$$

$$U = Q V_{:, \mathcal{K}}, \quad \lambda_i = |\lambda_{\text{raw},i}| \text{ (flip)}, \quad F \approx U \Lambda U^T, \quad U^T U = I_r \quad (9)$$

with

$$\text{cholqr2}(X): \quad G = X^T X = \sum_p X_p^T X_p \quad (\text{one allreduce}); \quad G = \hat{R}^T \hat{R}; \quad X \leftarrow X \hat{R}^{-1}; \quad \text{repeat once.} \quad (10)$$

2. GLR Hessian apply

$$\tilde{H} := ZM^{-1/2} (cI + U \Lambda U^T) M^{-1/2} Z \approx \tilde{H}_d + cR \quad (11)$$

$$\tilde{H}v: \quad v_1 = Zv \quad (\text{sparse matvec}) \quad (12)$$

$$v_2 = M^{-1/2}v_1 \quad (13)$$

$$w = U^T v_2 \quad (w \in \mathbb{R}^r) \quad (14)$$

$$v_3 = c v_2 + U(\Lambda w) \quad (15)$$

$$v_4 = M^{-1/2}v_3 \quad (16)$$

$$\tilde{H}v = Zv_4 \quad (17)$$

(`lgh_glr_apply` computes the exact $\tilde{H}_d v + cRv$ instead — no truncation error — but this spectral route is what `lgh_glr_filter` generalizes.)

3. GLR Hessian solve (Woodbury via orthonormality)

$$(cI + U\Lambda U^T)^{-1} = \frac{1}{c} (I - UDU^T), \quad D = \text{diag}\left(\frac{\lambda_i}{\lambda_i + c}\right) \quad (18)$$

$$\tilde{H}^{-1}r : \quad r_1 = Z^{-1}r \quad (\text{iterative Z-solve: Chebyshev/V-cycle}) \quad (19)$$

$$r_2 = M^{1/2}r_1 \quad (20)$$

$$w = U^T r_2 \quad (21)$$

$$r_3 = \frac{1}{c} (r_2 - U(Dw)) \quad (22)$$

$$r_4 = M^{1/2}r_3 \quad (23)$$

$$\tilde{H}^{-1}r = Z^{-1}r_4 \quad (24)$$

This is `lgh_glr_solve`.

Symbol shapes and memory distribution

Symbol	Shape	Space	Memory layout
v, r, v_i, r_i	N	function / dual	distributed by mesh partition (N_p per rank)
$M^{\pm 1/2}$	N (diag)	—	distributed by mesh partition
Z	$N \times N$ sparse	dual $\leftarrow$ function	distributed rows; applied matrix-free (AMG/Cheb.)
$\tilde{H}_d$	$N \times N$ sparse	dual $\leftarrow$ function	distributed rows (MPIAIJ)
Ω, Y, Q	$N \times \ell$ dense	whitened	distributed rows: contiguous $N_p \times \ell$ slab (Ω hashed from (seed, global i, j): drawn locally)
$G, \hat{R}$	$\ell \times \ell$	feature	replicated (one allreduce); redundant Cholesky
T	$\ell \times \ell$	feature	replicated (one allreduce); redundant <code>dsyevd</code>
V	$\ell \times \ell$	feature	replicated
λ, D	r (diag)	feature	replicated
U	$N \times r$ dense	whitened $\leftarrow$ feature	distributed rows: $N_p \times r$ slab
w	r	feature	replicated coefficient array (local GEMV + one allreduce inside $U^T x$)
c, τ, α	scalar	—	replicated

Representative production build from the engine's originating application (a warm build on a ~ 10 k-dof mesh): $N = 10371$, $\ell = 2200$, $q = 1$, $\tau = 1.0$, $r = 2136$.